Employee Data Pipeline Report

Name: Kalpesh Gadhari
Project: Employee Data Pipeline using Airflow, PostgreSQL & Docker

---

1. Project Overview

This project implements an **end to end ETL data pipeline** using Apache Airflow. The pipeline processes employee data from a CSV file, performs data cleaning and transformation, and loads the final structured data into a PostgreSQL database.

The goal of this project is to demonstrate **data engineering concepts such as data gatherings, data cleaning, deduplication, and containerization**.

---

2. Tech Stack

- **Apache Airflow** – Workflow orchestration
- **Python (Pandas)** – Data transformation
- **PostgreSQL** – Data storage
- **Docker** – Containerization
- **GitHub** – Version control

---

3. Pipeline Architecture

Flow:

CSV File → Airflow DAG → Data Cleaning (Pandas) → PostgreSQL

---

4. Implementation Details

4.1 Extract Phase

- Data is read from a CSV file (employees_raw.csv)
- Airflow task extracts and passes data using XCom

4.2 Transform Phase

Data cleaning operations performed:

- Removal of duplicate records based on employee_id
- Handling of missing values
- Data formatting and structuring

Key Transformation Logic:

- Used drop_duplicates(subset=['employee_id'])
- Ensured only unique employee records are retained

4.3 Load Phase

- Data is loaded into PostgreSQL table employees_clean
- Table is truncated before insert to avoid duplication
- Cleaned records are inserted row by row

---

5. Database Design

```
CREATE TABLE employees_clean (
  employee_id INTEGER PRIMARY KEY,
  first_name VARCHAR(50),
  last_name VARCHAR(50),
  full_name VARCHAR(100),
  email VARCHAR(100),
  email_domain VARCHAR(50),
  hire_date DATE,
  job_title VARCHAR(100),
  department VARCHAR(50),
  salary DECIMAL(10,2),
  salary_band VARCHAR(20),
  manager_id INTEGER,
  address TEXT,
  city VARCHAR(50),
  state VARCHAR(10),
  zip_code VARCHAR(10),
  birth_date DATE,
  age INTEGER,
  tenure_years DECIMAL(5,2),
  status VARCHAR(20)
);
```

Constraints:

- employee_id is unique (Primary Key)
- Ensures no duplicate entries in final dataset

---

6. Data Validation & Results

Row Count Check

```
SELECT COUNT(*) FROM employees_clean;
```

Output:
77 rows

Duplicate Check

SELECT employee_id, COUNT(*)

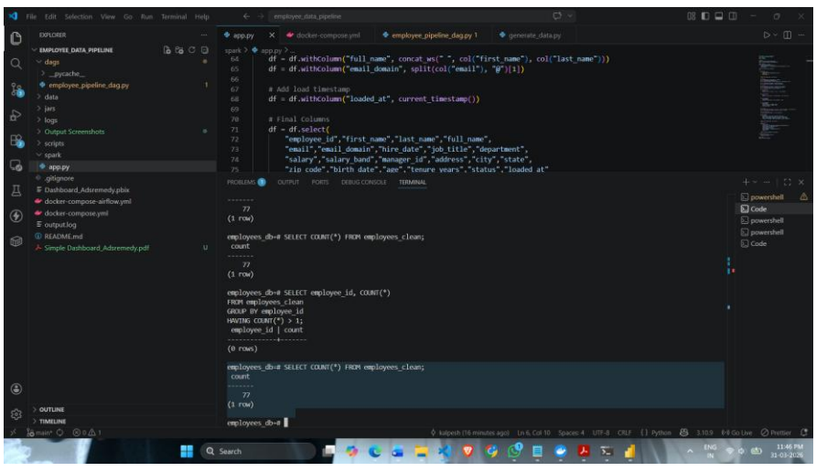
FROM employees_clean

GROUP BY employee_id

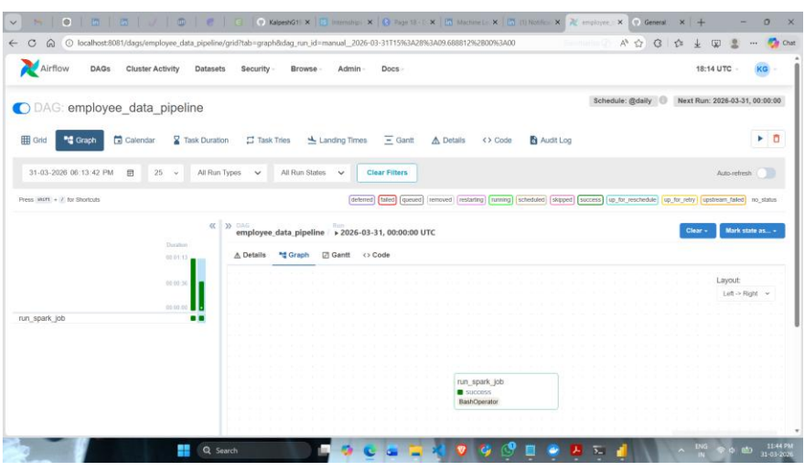
HAVING COUNT(*) > 1;

Output: No duplicate rows found

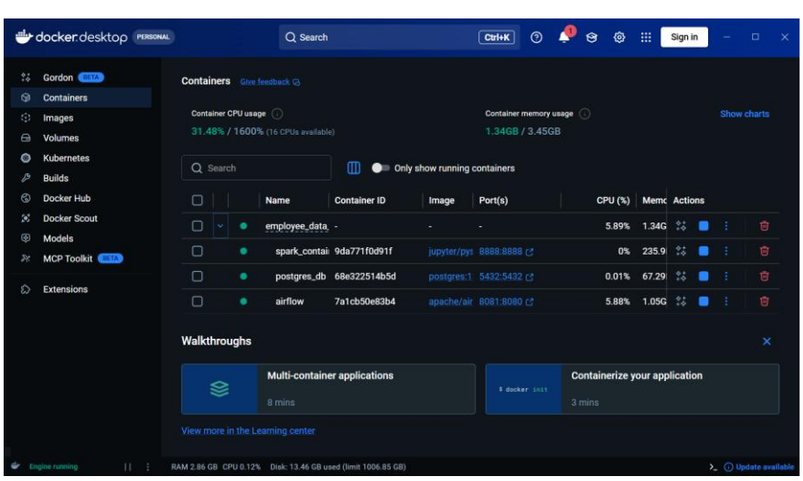
Output Screenshot



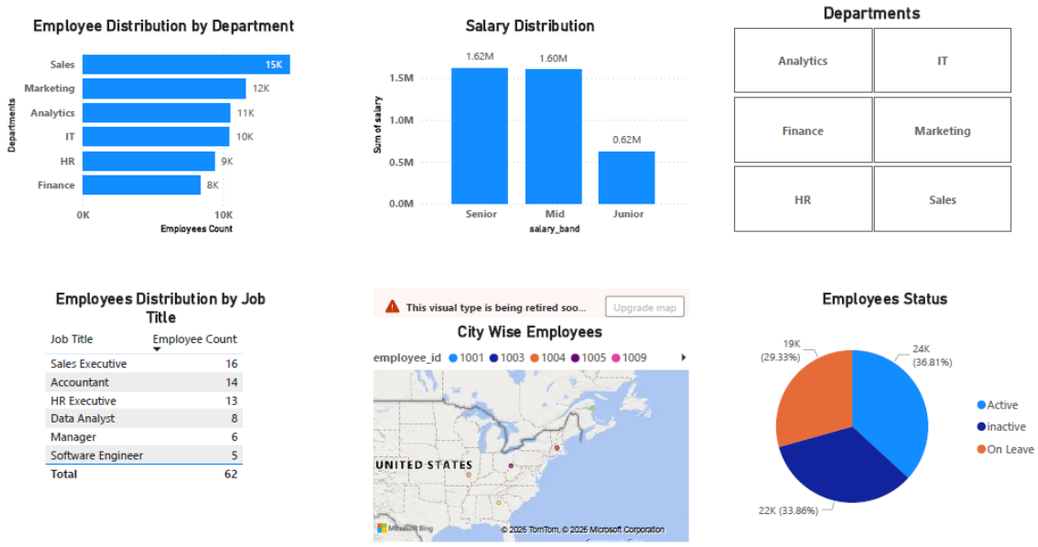
Airflow DAG Success



Docker Container



Simple Power BI Dashboard



---

7. Data Quality Improvements

Before transformation:

- Duplicate records present (~154 rows)

After transformation:

- Clean dataset with **77 unique records**
- No duplicates
- Structured and validated data

---

8. Docker & Infrastructure

The project uses Docker for containerized setup:

Services:

- Airflow (port 8081)
- PostgreSQL (port 5432)
- Spark (optional container)

Benefits:

- Easy setup
- Isolated environment
- Reproducibility

---

9. Challenges Faced

- Airflow UI not loading initially
- Database initialization issues
- Duplicate data handling
- Container communication setup

---

10. Solutions Implemented

- Fixed Airflow DB initialization using proper setup
- Implemented deduplication using Pandas
- Added TRUNCATE before insert to avoid duplication
- Validated data using SQL queries

---

11. Key Learnings

- Understanding of ETL pipeline design
- Hands on experience with Airflow DAGs
- Data cleaning and validation techniques
- Docker based deployment
- Debugging real world pipeline issues

---

12. Conclusion

This project successfully demonstrates a **ETL pipeline** with:

- Automated workflow orchestration
- Clean and validated data
- Scalable architecture using Docker
- Real world data engineering practices